

Driving School Management System — Final Database Specification V3

This document is the final database specification for the Driving School Management System. It is intended to be the single source of truth before implementing TypeORM entities, database migrations, services, and APIs.

1. Project Stack and Architecture

The backend system is built using:

- **NestJS**
- **TypeScript**
- **PostgreSQL**
- **TypeORM**
- **Docker Compose**
- **pgAdmin**
- **JWT + Passport**
- **bcrypt**
- **class-validator / class-transformer**

The project uses a practical modular architecture instead of full Clean Architecture.

This decision is intentional because:

- This is the team's first major NestJS + TypeORM project.
- The project has a limited timeline.
- The business requirements are mostly stable.
- The goal is to build a clean, maintainable, realistic backend without unnecessary abstraction.

Each module should follow this structure:

```
module/  
├─ module.controller.ts  
├─ module.service.ts  
├─ module.entity.ts  
├─ dto/  
└─ module.module.ts
```

General architecture rules:

- Controllers must stay thin.
- Business logic must live inside services.
- Request validation must be handled through DTOs.
- Authorization must be handled through guards.
- Errors must be standardized through a global exception filter.
- API responses can later be standardized using a response interceptor.
- Database schema changes must be handled through migrations.
- `TypeORM synchronize` must remain disabled.

2. General Database Conventions

2.1 Primary Keys

Every table uses:

```
id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
```

2.2 Foreign Keys

Foreign keys use the format:

```
<singular_entity_name>_id
```

Examples:

```
student_id
instructor_id
vehicle_id
booking_id
certificate_id
```

2.3 Time and Date Types

Use:

```
TIMESTAMPZ
```

for date + time values.

Use:

```
DATE
```

for date-only values.

Use:

```
TIME
```

for time-only values.

2.4 Money Values

All financial amounts use:

```
NUMERIC(10,2)
```

2.5 Soft Deactivation

Users, students, instructors, and employees are not physically deleted.

The user account is managed through:

```
users.account_status
```

Important relationships pointing to users, students, instructors, and employees should generally use:

```
ON DELETE RESTRICT
```

2.6 Enum Storage

Enums are stored in English uppercase codes in the database.

Examples:

```
ACTIVE  
BOOKED  
MANUAL  
AUTOMATIC  
PAID  
UNPAID
```

Arabic translation should be handled in the frontend.

3. RBAC and Profile Creation Rule

The system uses full RBAC:

```
roles  
permissions  
role_permissions  
user_roles
```

However, roles alone are not enough to define the real business profile of a user.

The backend must enforce the following rules in services using database transactions.

3.1 Creating a Student

When creating a student, the backend must do all of the following inside one transaction:

1. Create a row in users
2. Create a row in students
3. Assign role STUDENT to the user

3.2 Creating an Instructor

When creating an instructor, the backend must do all of the following inside one transaction:

1. Create a row in users
2. Create a row in instructors
3. Assign role INSTRUCTOR to the user

3.3 Creating an Employee

When creating an employee, the backend must do all of the following inside one transaction:

1. Create a row in users
2. Create a row in employees
3. Assign the correct employee role to the user

Examples:

MANAGER
RECEPTIONIST
ACCOUNTANT

The purpose of this rule is to prevent inconsistent data such as:

A user with STUDENT role but no students row
A user with INSTRUCTOR role but no instructors row
A student row without a STUDENT role
An employee row without an employee role

4. Table Count

This database specification contains:

36 tables

Main sections:

Users and access control
Bookings and vehicles
Payments and charges
Expenses
Government certificates
Transport service
Notifications
Authentication support
Instructor schedule publications
Settings

5. Tables

Section 1 — Users and Access Control

5.1 users

Represents the base account for any person who can exist in the system.

users

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
name	VARCHAR(100)	Yes	Full name
phone	VARCHAR(20)	Yes	Unique phone number
password_hash	VARCHAR(255)	Yes	Hashed password
must_change_password	BOOLEAN	Yes	Default true when created by staff
account_status	ENUM(ACTIVE, BLOCKED, ARCHIVED)	Yes	Account state
created_at	TIMESTAMPTZ	Yes	Default now()

Constraints:

UNIQUE(phone)

Notes:

- The password must never be stored as plain text.
- password_hash stores the bcrypt hash.
- Users are not physically deleted.
- Blocking or archiving happens through account_status .

5.2 roles

Represents system roles.

roles

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
title	ENUM(MANAGER, RECEPTIONIST, ACCOUNTANT, INSTRUCTOR, STUDENT)	Yes	Unique role title

Column	Type	Required	Notes
description	VARCHAR(255)	No	Role description

Constraints:

```
UNIQUE(title)
```

Role examples:

```
MANAGER
RECEPTIONIST
ACCOUNTANT
INSTRUCTOR
STUDENT
```

5.3 permissions

Represents fine-grained permissions.

```
permissions
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
code	VARCHAR(80)	Yes	Unique permission code
description	VARCHAR(255)	No	Permission description
module	VARCHAR(60)	No	Related application module

Constraints:

```
UNIQUE(code)
```

Examples:


```
users.create
users.read
users.update
bookings.create
bookings.cancel
payments.verify
vehicles.create
certificates.update
settings.update
```

5.4 role_permissions

Join table between roles and permissions.

```
role_permissions
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
role_id	BIGINT	Yes	FK → roles.id
permission_id	BIGINT	Yes	FK → permissions.id

Relationships:

```
roles 1 --- N role_permissions
permissions 1 --- N role_permissions
roles M --- N permissions
```

Delete behavior:

```
role_id ON DELETE CASCADE
permission_id ON DELETE CASCADE
```

Constraints:

```
UNIQUE(role_id, permission_id)
```

5.5 user_roles

Join table between users and roles.

user_roles

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
user_id	BIGINT	Yes	FK → users.id
role_id	BIGINT	Yes	FK → roles.id

Relationships:

users 1 --- N user_roles
roles 1 --- N user_roles
users M --- N roles

Delete behavior:

user_id ON DELETE CASCADE
role_id ON DELETE RESTRICT

Constraints:

UNIQUE(user_id, role_id)

5.6 activity_log

Stores important user actions.

activity_log

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
action	VARCHAR(120)	Yes	Action name
note	VARCHAR(255)	No	Short note
user_id	BIGINT	Yes	FK → users.id
created_at	TIMESTAMPTZ	Yes	Default now()

Relationships:

`users 1 --- N activity_log`

Delete behavior:

`user_id ON DELETE RESTRICT`

5.7 students

Represents student-specific data.

`students`

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
student_status	ENUM(PASSED, FAILED, IN_TRAINING, CERTIFICATE_SEEKER)	Yes	Student status
user_id	BIGINT	Yes	FK → users.id , unique

Relationships:

`users 1 --- 1 students`

Delete behavior:

```
user_id ON DELETE RESTRICT
```

Constraints:

```
UNIQUE(user_id)
```

Business rule:

When creating a student, the system must also assign role STUDENT to the related user.

5.8 instructors

Represents instructor-specific data.

```
instructors
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
gender	ENUM(MALE, FEMALE)	Yes	Instructor gender
instructor_type	ENUM(MANUAL, AUTOMATIC, BOTH)	Yes	Instructor training capability
user_id	BIGINT	Yes	FK → users.id , unique

Relationships:

```
users 1 --- 1 instructors
```

Delete behavior:

```
user_id ON DELETE RESTRICT
```

Constraints:

UNIQUE(user_id)

Business rule:

When creating an instructor, the system must also assign role INSTRUCTOR to the related us

5.9 employees

Represents administrative employees.

employees

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
user_id	BIGINT	Yes	FK → users.id , unique
hire_date	DATE	No	Hiring date
monthly_salary	NUMERIC(10,2)	No	Monthly salary
resign_date	DATE	No	Resignation or leaving date

Relationships:

users 1 --- 1 employees

Delete behavior:

user_id ON DELETE RESTRICT

Constraints:

UNIQUE(user_id)

Business rule:

When creating an employee, the system must assign an employee-related role such as `MANAGER`

Section 2 — Bookings and Vehicles

5.10 vehicles

Represents school vehicles.

`vehicles`

Column	Type	Required	Notes
<code>id</code>	<code>BIGINT</code>	Yes	Primary key
<code>plate_number</code>	<code>VARCHAR(20)</code>	Yes	Unique plate number
<code>model</code>	<code>VARCHAR(50)</code>	No	Vehicle model
<code>color</code>	<code>VARCHAR(30)</code>	No	Vehicle color
<code>type</code>	<code>ENUM(MANUAL, AUTOMATIC)</code>	Yes	Vehicle training type
<code>status</code>	<code>ENUM(ACTIVE, INACTIVE, ARCHIVED)</code>	Yes	Administrative status
<code>admin_notes</code>	<code>VARCHAR(255)</code>	No	Internal notes
<code>created_at</code>	<code>TIMESTAMPTZ</code>	Yes	Default <code>now()</code>

Constraints:

`UNIQUE(plate_number)`

Notes:

- `status` is an administrative state.
- A vehicle being currently used is determined by `booking` .
- Temporary vehicle unavailability is stored in `vehicle_unavailable_periods` .

5.11 booking

Represents a driving lesson booking.

booking

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
vehicle_source	ENUM(SCHOOL_CAR, STUDENT_CAR)	Yes	Vehicle source
booking_status	ENUM(PENDING_PAYMENT, BOOKED, COMPLETED, NO_SHOW, CANCELLED, EXPIRED)	Yes	Booking status
payment_status	ENUM(PENDING_DEPOSIT, DEPOSIT_PAID, FULLY_PAID, DEPOSIT_NON_REFUNDABLE, DEPOSIT_AVAILABLE_FOR_REBOOKING, DEPOSIT_USED_IN_REBOOKING)	Yes	Booking payment snapshot
training_type	ENUM(MANUAL, AUTOMATIC)	Yes	Training type only
start_at	TIMESTAMPTZ	Yes	Booking start time
end_at	TIMESTAMPTZ	Yes	Booking end time
locked_until	TIMESTAMPTZ	No	Temporary payment hold expiration
student_id	BIGINT	Yes	FK → students.id

Column	Type	Required	Notes
instructor_id	BIGINT	Yes	FK → instructors.id
vehicle_id	BIGINT	No	FK → vehicles.id
replaced_booking_id	BIGINT	No	FK → booking.id , unique
created_at	TIMESTAMPTZ	Yes	Default now()
updated_at	TIMESTAMPTZ	Yes	Updated on changes

Relationships:

```

students 1 --- N booking
instructors 1 --- N booking
vehicles 1 --- N booking
booking 1 --- 0..1 booking through replaced_booking_id

```

Delete behavior:

```

student_id ON DELETE RESTRICT
instructor_id ON DELETE RESTRICT
vehicle_id ON DELETE RESTRICT
replaced_booking_id ON DELETE RESTRICT

```

Constraints:

```
CHECK (start_at < end_at)
```

```

If vehicle_source = SCHOOL_CAR, vehicle_id must be NOT NULL.
If vehicle_source = STUDENT_CAR, vehicle_id must be NULL.

```

```
UNIQUE(replaced_booking_id)
```


Booking status meanings:

PENDING_PAYMENT = The student reserved a slot and has not paid yet.

BOOKED = Confirmed booking.

COMPLETED = Lesson completed.

NO_SHOW = Student did not attend.

CANCELLED = Booking manually cancelled.

EXPIRED = Payment hold expired before confirmation.

Important booking rules:

- PENDING_PAYMENT temporarily reserves the instructor and vehicle until `locked_until`.
- BOOKED means the booking is confirmed.
- EXPIRED rows are kept to track unpaid attempts and prevent abuse.
- Availability queries must ignore EXPIRED bookings.

5.12 vehicle_unavailable_periods

Represents periods where a school vehicle is unavailable for reasons other than bookings.

vehicle_unavailable_periods

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
vehicle_id	BIGINT	Yes	FK → vehicles.id
start_at	TIMESTAMPTZ	Yes	Unavailability start time
end_at	TIMESTAMPTZ	No	Unavailability end time
reason_type	ENUM(MAINTENANCE, UNAVAILABLE, OUT_OF_SERVICE)	Yes	Reason type
notes	VARCHAR(255)	No	Notes
created_at	TIMESTAMPTZ	Yes	Default now()

Relationships:

```
vehicles 1 --- N vehicle_unavailable_periods
```

Delete behavior:

```
vehicle_id ON DELETE RESTRICT
```

Notes:

```
vehicles.status = general administrative vehicle status  
vehicle_unavailable_periods = temporary vehicle unavailability  
booking = vehicle usage by bookings
```

5.13 booking_cancellations

Stores cancellation details for cancelled bookings.

```
booking_cancellations
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
cancellation_party	ENUM(STUDENT, VEHICLE, INSTRUCTOR, SCHOOL)	Yes	Party responsible for cancellation
cancellation_reason	VARCHAR(255)	No	Cancellation reason
cancelled_at	TIMESTAMPTZ	Yes	Cancellation time
booking_id	BIGINT	Yes	FK → booking.id , unique
cancelled_by_user_id	BIGINT	No	FK → users.id

Relationships:

```
booking 1 --- 0..1 booking_cancellations
users 1 --- N booking_cancellations through cancelled_by_user_id
```

Delete behavior:

```
booking_id ON DELETE CASCADE
cancelled_by_user_id ON DELETE RESTRICT
```

Constraints:

```
UNIQUE(booking_id)
```

Notes:

- A booking can only have one cancellation record.
- `cancellation_party` explains the business cause.
- `cancelled_by_user_id` stores the actual user who performed the cancellation action.

Example:

```
cancellation_party = SCHOOL
cancelled_by_user_id = the receptionist user who cancelled the booking
```

5.14 lesson_price

Represents driving lesson prices.

```
lesson_price
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
instructor_gender	ENUM(MALE, FEMALE)	Yes	Instructor gender
training_type	ENUM(MANUAL, AUTOMATIC)	Yes	Training type
vehicle_source	ENUM(SCHOOL_CAR, STUDENT_CAR)	Yes	Vehicle source

Column	Type	Required	Notes
price	NUMERIC(10,2)	Yes	Lesson price
effective_from	DATE	Yes	Price start date
created_at	TIMESTAMPTZ	Yes	Default now()

Constraints:

```
UNIQUE(instructor_gender, training_type, vehicle_source, effective_from)
```

Price lookup dimensions:

```
Instructor gender
Training type
Vehicle source
Effective date
```

Business rule:

```
When creating booking charges, the system reads the currently effective price and copies i
```

This keeps old charges stable even if prices change later.

5.15 instructor_weekly_availabilities

Represents weekly instructor working hours.

```
instructor_weekly_availabilities
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
day_of_week	ENUM(SAT, SUN, MON, TUE, WED, THU, FRI)	Yes	Day of week
start_time	TIME	Yes	Work start time

Column	Type	Required	Notes
end_time	TIME	Yes	Work end time
instructor_id	BIGINT	Yes	FK → instructors.id

Relationships:

```
instructors 1 --- N instructor_weekly_availabilities
```

Delete behavior:

```
instructor_id ON DELETE CASCADE
```

Constraints:

```
UNIQUE(instructor_id, day_of_week, start_time)
CHECK (start_time < end_time)
```

Business rule:

```
The system should prevent overlapping weekly availability periods for the same instructor
```

5.16 instructor_unavailable_periods

Represents periods where an instructor is unavailable.

```
instructor_unavailable_periods
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
instructor_id	BIGINT	Yes	FK → instructors.id
start_at	TIMESTAMPTZ	Yes	Unavailability start time

Column	Type	Required	Notes
end_at	TIMESTAMPTZ	Yes	Unavailability end time
reason	VARCHAR(255)	No	Reason
created_at	TIMESTAMPTZ	Yes	Default now()

Relationships:

```
instructors 1 --- N instructor_unavailable_periods
```

Delete behavior:

```
instructor_id ON DELETE CASCADE
```

Constraints:

```
CHECK (start_at < end_at)
```

Notes:

This table supports:

```
Full-day leave
Partial-day emergency
Administrative block
Instructor absence
```

Section 3 — Charges and Payments

5.17 student_charges

Represents amounts that a student owes.

student_charges

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
charge_reason	ENUM(LESSON_DEPOSIT, LESSON_REMAINDER, CERTIFICATE_FEE, TRANSPORT_LECTURE, TRANSPORT_EXAM, REEXAM_THEORY, REEXAM_PRACTICAL, OTHER)	Yes	Charge reason
amount_due	NUMERIC(10,2)	Yes	Amount due
charge_status	ENUM(PAID, UNPAID, PARTIALLY_PAID, CANCELLED)	Yes	Charge status
created_at	TIMESTAMPTZ	Yes	Default now()
due_at	DATE	No	Due date
student_id	BIGINT	Yes	FK → students.id
booking_id	BIGINT	No	FK → booking.id
certificate_id	BIGINT	No	FK → certificates.id
certificate_exam_result_id	BIGINT	No	FK → certificate_exam_results.id

Relationships:

```

students 1 --- N student_charges
booking 1 --- N student_charges
certificates 1 --- N student_charges
certificate_exam_results 1 --- N student_charges

```

Delete behavior:

student_id ON DELETE RESTRICT
booking_id ON DELETE RESTRICT
certificate_id ON DELETE RESTRICT
certificate_exam_result_id ON DELETE RESTRICT

Business source rules:

LESSON_DEPOSIT and LESSON_REMAINDER require booking_id.
CERTIFICATE_FEE requires certificate_id.
REEXAM_THEORY and REEXAM_PRACTICAL require certificate_exam_result_id.
TRANSPORT_LECTURE and TRANSPORT_EXAM are connected to transport logic.

Payment rule:

A single charge can have multiple payments.

This supports partial payments.

5.18 student_payments

Represents actual money received by the school.

student_payments

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
amount_paid	NUMERIC(10,2)	Yes	Amount paid
payment_method	ENUM(CASH, SHAM_CASH)	Yes	Payment method
received_at	TIMESTAMPTZ	Yes	Payment received time
student_charge_id	BIGINT	Yes	FK → student_charges.id

Relationships:

student_charges 1 --- N student_payments

Delete behavior:

```
student_charge_id ON DELETE RESTRICT
```

Charge status calculation:

```
If total paid = 0:  
charge_status = UNPAID
```

```
If total paid > 0 and total paid < amount_due:  
charge_status = PARTIALLY_PAID
```

```
If total paid >= amount_due:  
charge_status = PAID
```

5.19 shamcash_transactions

Represents verified ShamCash transactions.

```
shamcash_transactions
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
transaction_id	VARCHAR(100)	Yes	Unique ShamCash transaction ID
sender_account	VARCHAR(50)	No	Sender account
receiver_account	VARCHAR(50)	No	School receiver account
amount	NUMERIC(10,2)	Yes	Amount returned by ShamCash
occurred_at	TIMESTAMPTZ	No	Transaction occurrence time from ShamCash
verified_at	TIMESTAMPTZ	Yes	Verification time in our system
raw_payload	JSONB	No	Raw ShamCash API response
student_payment_id	BIGINT	Yes	FK → student_payments.id, unique

Relationships:

```
student_payments 1 --- 0..1 shamcash_transactions
```

Delete behavior:

```
student_payment_id ON DELETE CASCADE
```

Constraints:

```
UNIQUE(transaction_id)
UNIQUE(student_payment_id)
```

Business rules:

Only payments with payment_method = SHAM_CASH should have a shamcash_transactions row.
Cash payments do not have a shamcash_transactions row.
transaction_id must be unique to prevent reuse.

Section 4 — Expenses

5.20 expenses

Central expense ledger.

```
expenses
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
category	ENUM(VEHICLE, INSTRUCTOR, EMPLOYEE, GENERAL)	Yes	Expense category

Column	Type	Required	Notes
amount	NUMERIC(10,2)	Yes	Expense amount
expense_date	DATE	Yes	Expense or payment date
status	ENUM(PAID, UNPAID)	Yes	Expense status
note	VARCHAR(255)	No	Note
created_at	TIMESTAMPTZ	Yes	Default now()
created_by_employee_id	BIGINT	No	FK → employees.id

Relationships:

employees 1 --- N expenses through created_by_employee_id

Delete behavior:

created_by_employee_id ON DELETE RESTRICT

Notes:

- expenses is the central table used for financial reports.
- Expense-specific details are stored in related subtype tables.

5.21 expenses_vehicle

Vehicle expense details.

expenses_vehicle

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key

Column	Type	Required	Notes
reason	ENUM(MAINTENANCE, GAS, INSURANCE, OTHER)	Yes	Vehicle expense reason
liters	NUMERIC(6,2)	No	Fuel liters if applicable
vehicle_id	BIGINT	Yes	FK → vehicles.id
expense_id	BIGINT	Yes	FK → expenses.id , unique

Relationships:

```
expenses 1 --- 1 expenses_vehicle
vehicles 1 --- N expenses_vehicle
```

Delete behavior:

```
expense_id ON DELETE CASCADE
vehicle_id ON DELETE RESTRICT
```

Constraints:

```
UNIQUE(expense_id)
```

5.22 expenses_instructor

Instructor earning expense details.

```
expenses_instructor
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
booking_id	BIGINT	Yes	FK → booking.id , unique
expense_id	BIGINT	Yes	FK → expenses.id , unique

Relationships:

```
booking 1 --- 0..1 expenses_instructor
expenses 1 --- 1 expenses_instructor
```

Delete behavior:

```
booking_id ON DELETE RESTRICT
expense_id ON DELETE CASCADE
```

Constraints:

```
UNIQUE(booking_id)
UNIQUE(expense_id)
```

Business rule:

An `expenses_instructor` row is created only when `booking_status` becomes `COMPLETED`.

5.23 expenses_employee

Employee expense details.

`expenses_employee`

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
type	ENUM(SALARY, BONUS, OTHER)	Yes	Employee expense type
month	DATE	Yes	Salary or bonus month
employee_id	BIGINT	Yes	FK → employees.id
expense_id	BIGINT	Yes	FK → expenses.id , unique

Relationships:

```
employees 1 --- N expenses_employee
expenses 1 --- 1 expenses_employee
```

Delete behavior:

```
employee_id ON DELETE RESTRICT
expense_id ON DELETE CASCADE
```

Constraints:

```
UNIQUE(expense_id)
```

5.24 expenses_general

General expense details.

```
expenses_general
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
type	ENUM(WATER, ELECTRICITY, INTERNET, KITCHEN, SUPPLIES, OTHER)	Yes	General expense type
expense_id	BIGINT	Yes	FK → expenses.id , unique

Relationships:

```
expenses 1 --- 1 expenses_general
```

Delete behavior:

```
expense_id ON DELETE CASCADE
```

Constraints:

UNIQUE(expense_id)

5.25 instructor_price

Represents instructor earning rate per completed lesson.

instructor_price

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
type	ENUM(MALE, FEMALE)	Yes	Instructor gender
price	NUMERIC(10,2)	Yes	Instructor earning amount
effective_from	DATE	Yes	Rate effective date

Constraints:

UNIQUE(type, effective_from)

Business rule:

When a booking is completed, the system reads the effective instructor price by gender and

Section 5 — Government Certificates

5.26 certificates

Represents a government certificate request or license category upgrade.

certificates

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
category	ENUM(B, B1, C, D)	Yes	Certificate category
request_status	ENUM(WAITING_FOR_SERVICE_FEE, WAITING_FOR_TRAINING_SCHEDULE, IN_GOVERNMENT_TRAINING, WAITING_FOR_PRACTICAL_EXAM, WAITING_FOR_THEORETICAL_EXAM, COMPLETED, CANCELLED)	Yes	Request workflow status
personal_photo_url	VARCHAR(500)	Yes	Personal photo URL
transport_requested	BOOLEAN	Yes	Whether transport is requested
requested_at	TIMESTAMPTZ	Yes	Request creation time
student_id	BIGINT	Yes	FK → students.id

Relationships:

students 1 --- N certificates

Delete behavior:

student_id ON DELETE RESTRICT

Workflow statuses:

WAITING_FOR_SERVICE_FEE = Waiting for certificate service fee payment
WAITING_FOR_TRAINING_SCHEDULE = Waiting for government training schedule
IN_GOVERNMENT_TRAINING = Student is in government training
WAITING_FOR_PRACTICAL_EXAM = Waiting for practical exam
WAITING_FOR_THEORETICAL_EXAM = Waiting for theoretical exam
COMPLETED = Certificate workflow completed
CANCELLED = Certificate request cancelled

5.27 certificate_training_sessions

Represents government training sessions.

certificate_training_sessions

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
session_number	SMALLINT	Yes	Session number
scheduled_at	TIMESTAMPTZ	No	Session schedule
certificate_id	BIGINT	Yes	FK → certificates.id

Relationships:

certificates 1 --- N certificate_training_sessions

Delete behavior:

certificate_id ON DELETE CASCADE

Constraints:

UNIQUE(certificate_id, session_number)

Business rule:

A certificate request usually has 3 government training sessions.

5.28 certificate_exam_results

Represents certificate exam attempts and results.

certificate_exam_results

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
exam_type	ENUM(THEORY, PRACTICAL)	Yes	Exam type
attempt_number	SMALLINT	Yes	Attempt number
scheduled_at	TIMESTAMPTZ	No	Exam schedule
exam_result	ENUM(PASS, FAIL, ABSENT)	No	Exam result
result_recorded_at	TIMESTAMPTZ	No	Result recording time
certificate_id	BIGINT	Yes	FK → certificates.id

Relationships:

certificates 1 --- N certificate_exam_results

Delete behavior:

certificate_id ON DELETE CASCADE

Constraints:

UNIQUE(certificate_id, exam_type, attempt_number)

Business rules:

ABSENT is kept to distinguish absence from failure.
ABSENT can still be treated financially like FAIL if a re-exam fee is required.
When a re-exam fee is created, the charge must reference `certificate_exam_result_id`.

Section 6 — Transport Service

Transport service is not part of the core first implementation scope, but the schema is prepared for future support.

5.29 transport_trips

Represents group transport trips.

transport_trips

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
trip_type	ENUM(LECTURE, EXAM)	Yes	Trip type
trip_date	DATE	Yes	Trip date
day_number	SMALLINT	No	Lecture day number
assembly_time	TIME	No	Assembly time
destination	VARCHAR(150)	No	Destination
capacity	INTEGER	No	Trip capacity
status	ENUM(SCHEDULED, COMPLETED, CANCELLED)	Yes	Trip status
created_by	BIGINT	No	FK → employees.id

Column	Type	Required	Notes
created_at	TIMESTAMPTZ	Yes	Default now()

Relationships:

employees 1 --- N transport_trips

Delete behavior:

created_by ON DELETE RESTRICT

5.30 transport_registrations

Represents a certificate student registration for transport service.

transport_registrations

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
trip_type	ENUM(LECTURE, EXAM)	Yes	Transport registration type
registration_status	ENUM(REGISTERED, COMPLETED, CANCELLED)	Yes	Registration status
certificate_id	BIGINT	Yes	FK → certificates.id
student_charge_id	BIGINT	No	FK → student_charges.id, unique
created_by	BIGINT	No	FK → employees.id
created_at	TIMESTAMPTZ	Yes	Default now()

Relationships:

```
certificates 1 --- N transport_registrations
student_charges 1 --- 0..1 transport_registrations
employees 1 --- N transport_registrations
```

Delete behavior:

```
certificate_id ON DELETE RESTRICT
student_charge_id ON DELETE RESTRICT
created_by ON DELETE RESTRICT
```

Constraints:

```
UNIQUE(student_charge_id)
```

Notes:

Transport fees are represented through student_charges.

5.31 transport_attendance

Represents attendance of a transport registration in a specific trip.

```
transport_attendance
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
attendance_status	ENUM(PENDING, ATTENDED, ABSENT)	Yes	Attendance status
attended_at	TIMESTAMPTZ	No	Attendance time
transport_registration_id	BIGINT	Yes	FK → transport_registrations.id
transport_trip_id	BIGINT	Yes	FK → transport_trips.id

Relationships:

```
transport_registrations 1 --- N transport_attendance
transport_trips 1 --- N transport_attendance
transport_registrations M --- N transport_trips
```

Delete behavior:

```
transport_registration_id ON DELETE CASCADE
transport_trip_id ON DELETE RESTRICT
```

Constraints:

```
UNIQUE(transport_registration_id, transport_trip_id)
```

Section 7 — Notifications

5.32 notifications

Represents notifications sent to users.

```
notifications
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
recipient_user_id	BIGINT	Yes	FK → users.id
title	VARCHAR(150)	Yes	Notification title
body	TEXT	Yes	Notification content

Column	Type	Required	Notes
notification_type	ENUM(BOOKING_CONFIRMED, BOOKING_CANCELLED, BOOKING_EXPIRED, PAYMENT_ACCEPTED, PAYMENT_REJECTED, CERTIFICATE_STATUS_CHANGED, INSTRUCTOR_SCHEDULE, GENERAL)	Yes	Notification type
channel	ENUM(IN_APP, SMS, PUSH)	Yes	Delivery channel
status	ENUM(PENDING, SENT, FAILED, READ)	Yes	Notification status
sent_at	TIMESTAMPTZ	No	When the notification was sent
read_at	TIMESTAMPTZ	No	When the notification was read
created_at	TIMESTAMPTZ	Yes	Default now()

Relationships:

users 1 --- N notifications

Delete behavior:

recipient_user_id ON DELETE RESTRICT

Notes:

- The first implementation can use only `IN_APP` .
- `SMS` and `PUSH` are kept as future channels.
- No generic related entity fields are included in this version to keep the table simple.

Common use cases:

Booking confirmed
Booking cancelled
Booking expired
Payment accepted
Payment rejected
Certificate status changed
Instructor schedule sent
General administrative notification

Section 8 — Authentication Support

5.33 auth_otp_codes

Stores OTP codes for login, phone verification, and password reset.

auth_otp_codes

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
phone	VARCHAR(20)	Yes	Phone number receiving the OTP
user_id	BIGINT	No	FK → users.id
code_hash	VARCHAR(255)	Yes	Hashed OTP code
purpose	ENUM(STUDENT_LOGIN, PHONE_VERIFICATION, PASSWORD_RESET)	Yes	OTP purpose
expires_at	TIMESTAMPTZ	Yes	Expiration time
consumed_at	TIMESTAMPTZ	No	Time when OTP was used

Column	Type	Required	Notes
attempts_count	INTEGER	Yes	Number of verification attempts
created_at	TIMESTAMPTZ	Yes	Default now()

Relationships:

users 1 --- N auth_otp_codes

Delete behavior:

user_id ON DELETE CASCADE

Security rules:

The OTP code must not be stored as plain text.
code_hash stores the hashed OTP.
Expired OTP codes must not be accepted.
Consumed OTP codes must not be accepted again.
attempts_count should be limited by application logic.

Common purposes:

STUDENT_LOGIN = OTP login for student mobile app
PHONE_VERIFICATION = phone verification
PASSWORD_RESET = reset password flow

5.34 auth_sessions

Stores refresh-token sessions for authenticated users.

auth_sessions

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
user_id	BIGINT	Yes	FK → users.id
refresh_token_hash	VARCHAR(255)	Yes	Hashed refresh token
device_name	VARCHAR(120)	No	Device name or label
ip_address	VARCHAR(80)	No	IP address
user_agent	TEXT	No	Client user agent
expires_at	TIMESTAMPTZ	Yes	Session expiration
revoked_at	TIMESTAMPTZ	No	Session revocation time
created_at	TIMESTAMPTZ	Yes	Default now()

Relationships:

users 1 --- N auth_sessions

Delete behavior:

user_id ON DELETE CASCADE

Security rules:

Refresh tokens must not be stored as plain text.
 Only refresh_token_hash is stored.
 A session is active when revoked_at IS NULL and expires_at > now().
 Logout should set revoked_at.
 Changing password can revoke existing sessions.

Use cases:

Mobile app session
 Admin dashboard session
 Logout from one device
 Revoke all sessions after password change

Section 9 — Instructor Schedule Publications

5.35 instructor_schedule_publications

Stores daily instructor schedule publication records.

```
instructor_schedule_publications
```

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
instructor_id	BIGINT	Yes	FK → instructors.id
schedule_date	DATE	Yes	Date of the published schedule
published_at	TIMESTAMPTZ	Yes	Publication time
first_published_booking_start_at	TIMESTAMPTZ	No	First booking start time included in the published schedule
created_by_employee_id	BIGINT	No	FK → employees.id

Relationships:

```
instructors 1 --- N instructor_schedule_publications
employees 1 --- N instructor_schedule_publications through created_by_employee_id
```

Delete behavior:

```
instructor_id ON DELETE CASCADE
created_by_employee_id ON DELETE RESTRICT
```

Constraints:

UNIQUE(instructor_id, schedule_date)

Purpose:

This table records that an instructor schedule for a specific day was published.

Business rule:

After a schedule is published, the system can prevent new student bookings that make the i

Example:

Schedule for 2026-06-22 was published at 21:00.
The first published booking starts at 10:30.
The student app should not show a new 09:00 booking for that instructor after publication.

Section 10 — Settings

5.36 settings

Stores configurable system settings.

settings

Column	Type	Required	Notes
id	BIGINT	Yes	Primary key
key	VARCHAR(80)	Yes	Unique setting key
value	VARCHAR(255)	Yes	Value stored as text

Column	Type	Required	Notes
value_type	ENUM(NUMBER, STRING, TIME, PERCENT, BOOLEAN)	Yes	Value type
description	VARCHAR(255)	No	Setting description
updated_at	TIMESTAMPTZ	Yes	Last update time

Constraints:

`UNIQUE(key)`

Initial seed examples:

```
deposit_percentage = 50
booking_hold_minutes = 15
lesson_duration_minutes = 90
booking_window_days = 4
schedule_send_time = 21:00
transport_lecture_fee
transport_exam_fee
certificate_fee
reexam_theory_fee
reexam_practical_fee
```

Notes:

- Settings allow changing business values without editing code.
- Values are stored as strings and interpreted using `value_type` .

6. Booking and Availability Logic

6.1 Availability Generation

Available slots are not stored in a table.

They are generated dynamically from:

```
instructor_weekly_availabilities
instructor_unavailable_periods
booking
vehicles
vehicle_unavailable_periods
instructor_schedule_publications
settings
```

Availability generation steps:

1. Get instructors matching filters.
2. Generate time slots from instructor weekly availability.
3. Remove slots overlapping instructor_unavailable_periods.
4. Remove slots overlapping active bookings.
5. Apply instructor schedule publication rules if needed.
6. If vehicle_source = SCHOOL_CAR:
 - Find active vehicles matching the training type.
 - Remove vehicles unavailable through vehicle_unavailable_periods.
 - Remove vehicles already used in active bookings.
7. If vehicle_source = STUDENT_CAR:
 - Skip school vehicle checks.
8. Return available slots.

6.2 Booking Statuses That Reserve Time

The following statuses reserve instructor and vehicle time:

```
PENDING_PAYMENT
BOOKED
```

But PENDING_PAYMENT reserves time only while:

```
locked_until > now()
```

The following statuses do not reserve future availability:

EXPIRED
CANCELLED
COMPLETED
NO_SHOW

6.3 Booking Creation Flow

When a student presses "Book":

Inside a transaction:

1. Check whether the student has too many expired attempts today.
2. Check whether the student already has an active PENDING_PAYMENT booking.
3. Check instructor availability.
4. Check active booking conflicts.
5. If vehicle_source = SCHOOL_CAR:
 - Check vehicle availability.
 - Assign a suitable vehicle.
6. Create booking with booking_status = PENDING_PAYMENT.
7. Set locked_until = now() + settings.booking_hold_minutes.
8. Create LESSON_DEPOSIT student_charge.

When payment is verified:

1. Create student_payment.
2. If payment_method = SHAM_CASH, create shamcash_transactions.
3. Recalculate student_charge status.
4. If the deposit is paid:
 - booking_status = BOOKED
 - payment_status = DEPOSIT_PAID

When payment hold expires:

1. A scheduled job finds PENDING_PAYMENT bookings where locked_until < now().
2. booking_status becomes EXPIRED.
3. The slot becomes available again.
4. The EXPIRED row remains for abuse prevention and reporting.

Suggested abuse prevention rule:

If a student has 2 EXPIRED bookings today, prevent creating another PENDING_PAYMENT booking

7. Concurrency and Indexes

7.1 Time Overlap Rule

The generic overlap rule is:

```
existing_start < new_end AND existing_end > new_start
```

This rule applies to:

- Bookings
- Instructor unavailable periods
- Vehicle unavailable periods

7.2 PostgreSQL Exclusion Constraints

For strong protection against overlapping active bookings, PostgreSQL exclusion constraints are recommended.

Enable extension:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

Instructor overlap protection:


```
ALTER TABLE booking
ADD CONSTRAINT ex_booking_instructor_overlap
EXCLUDE USING gist (
    instructor_id WITH =,
    tstzrange(start_at, end_at, '[]') WITH &&
)
WHERE (
    booking_status IN ('PENDING_PAYMENT', 'BOOKED')
);
```

Vehicle overlap protection:

```
ALTER TABLE booking
ADD CONSTRAINT ex_booking_vehicle_overlap
EXCLUDE USING gist (
    vehicle_id WITH =,
    tstzrange(start_at, end_at, '[]') WITH &&
)
WHERE (
    vehicle_id IS NOT NULL
    AND booking_status IN ('PENDING_PAYMENT', 'BOOKED')
);
```

Notes:

- These constraints are stronger than `UNIQUE(instructor_id, start_at)`.
- They prevent real time overlaps, not only identical start times.
- Application-level transactions are still required.

7.3 Recommended Indexes

users(phone)
user_roles(user_id)
role_permissions(role_id)

booking(student_id)
booking(instructor_id, start_at, end_at)
booking(vehicle_id, start_at, end_at)
booking(booking_status)
booking(locked_until)

booking_cancellations(booking_id)
booking_cancellations(cancelled_by_user_id)

instructor_weekly_availabilities(instructor_id, day_of_week)
instructor_unavailable_periods(instructor_id, start_at, end_at)

vehicles(type, status)
vehicle_unavailable_periods(vehicle_id, start_at, end_at)

lesson_price(instructor_gender, training_type, vehicle_source, effective_from)

student_charges(student_id)
student_charges(charge_status)
student_charges(booking_id)
student_charges(certificate_id)
student_charges(certificate_exam_result_id)

student_payments(student_charge_id)
shamcash_transactions(transaction_id)
shamcash_transactions(student_payment_id)

expenses(expense_date)
expenses(category)
expenses(status)

certificates(student_id)
certificate_training_sessions(certificate_id, session_number)
certificate_exam_results(certificate_id, exam_type, attempt_number)

transport_registrations(certificate_id)
transport_attendance(transport_trip_id)

transport_attendance(transport_registration_id)

notifications(recipient_user_id)

notifications(status)

notifications(notification_type)

auth_otp_codes(phone)

auth_otp_codes(user_id)

auth_otp_codes(expires_at)

auth_sessions(user_id)

auth_sessions(expires_at)

auth_sessions(revoked_at)

instructor_schedule_publications(instructor_id, schedule_date)

settings(key)

8. Enum Reference

8.1 User and Access Control Enums

account_status:

ACTIVE

BLOCKED

ARCHIVED

roles.title:

MANAGER

RECEPTIONIST

ACCOUNTANT

INSTRUCTOR

STUDENT

```
student_status:  
PASSED  
FAILED  
IN_TRAINING  
CERTIFICATE_SEEKER
```

```
gender:  
MALE  
FEMALE
```

```
instructor_type:  
MANUAL  
AUTOMATIC  
BOTH
```

8.2 Booking and Vehicle Enums

```
vehicles.type:  
MANUAL  
AUTOMATIC
```

```
vehicles.status:  
ACTIVE  
INACTIVE  
ARCHIVED
```

```
vehicle_source:  
SCHOOL_CAR  
STUDENT_CAR
```

booking_status:

PENDING_PAYMENT

BOOKED

COMPLETED

NO_SHOW

CANCELLED

EXPIRED

payment_status:

PENDING_DEPOSIT

DEPOSIT_PAID

FULLY_PAID

DEPOSIT_NON_REFUNDABLE

DEPOSIT_AVAILABLE_FOR_REBOOKING

DEPOSIT_USED_IN_REBOOKING

training_type:

MANUAL

AUTOMATIC

vehicle_unavailable_reason_type:

MAINTENANCE

UNAVAILABLE

OUT_OF_SERVICE

8.3 Charge and Payment Enums

charge_reason:

LESSON_DEPOSIT

LESSON_REMAINDER

CERTIFICATE_FEE

TRANSPORT_LECTURE

TRANSPORT_EXAM

REEXAM_THEORY

REEXAM_PRACTICAL

OTHER

charge_status:

PAID

UNPAID

PARTIALLY_PAID

CANCELLED

payment_method:

CASH

SHAM_CASH

8.4 Expense Enums

expenses.category:

VEHICLE

INSTRUCTOR

EMPLOYEE

GENERAL

expenses.status:

PAID

UNPAID

expenses_vehicle.reason:

MAINTENANCE

GAS

INSURANCE

OTHER

expenses_employee.type:

SALARY

BONUS

OTHER

expenses_general.type:

WATER

ELECTRICITY

INTERNET

KITCHEN

SUPPLIES

OTHER

8.5 Certificate Enums

certificates.category:

B

B1

C

D

certificate request_status:

WAITING_FOR_SERVICE_FEE

WAITING_FOR_TRAINING_SCHEDULE

IN_GOVERNMENT_TRAINING

WAITING_FOR_PRACTICAL_EXAM

WAITING_FOR_THEORETICAL_EXAM

COMPLETED

CANCELLED

exam_type:

THEORY

PRACTICAL

exam_result:

PASS

FAIL

ABSENT

8.6 Transport Enums

trip_type:

LECTURE

EXAM

transport_trips.status:

SCHEDULED

COMPLETED

CANCELLED

registration_status:

REGISTERED

COMPLETED

CANCELLED

attendance_status:

PENDING

ATTENDED

ABSENT

8.7 Notification Enums

notification_type:

BOOKING_CONFIRMED

BOOKING_CANCELLED

BOOKING_EXPIRED

PAYMENT_ACCEPTED

PAYMENT_REJECTED

CERTIFICATE_STATUS_CHANGED

INSTRUCTOR_SCHEDULE

GENERAL


```
notification_channel:  
IN_APP  
SMS  
PUSH
```

```
notification_status:  
PENDING  
SENT  
FAILED  
READ
```

8.8 Authentication Enums

```
otp_purpose:  
STUDENT_LOGIN  
PHONE_VERIFICATION  
PASSWORD_RESET
```

8.9 Settings Enums

```
settings.value_type:  
NUMBER  
STRING  
TIME  
PERCENT  
BOOLEAN
```

9. Initial Production Seed Data

The first production seed should create:

1. One Super Admin user
2. Base roles
3. Base permissions
4. role_permissions mapping
5. settings rows
6. initial lesson_price rows
7. initial instructor_price rows

Base roles:

MANAGER
RECEPTIONIST
ACCOUNTANT
INSTRUCTOR
STUDENT

Base settings:

deposit_percentage = 50
booking_hold_minutes = 15
lesson_duration_minutes = 90
booking_window_days = 4
schedule_send_time = 21:00
transport_lecture_fee
transport_exam_fee
certificate_fee
reexam_theory_fee
reexam_practical_fee

10. Recommended Implementation Order

1. users
2. roles
3. permissions
4. role_permissions
5. user_roles
6. students
7. instructors
8. employees
9. auth_otp_codes
10. auth_sessions
11. vehicles
12. instructor_weekly_availabilities
13. instructor_unavailable_periods
14. vehicle_unavailable_periods
15. lesson_price
16. booking
17. booking_cancellations
18. student_charges
19. student_payments
20. shamcash_transactions
21. certificates
22. certificate_training_sessions
23. certificate_exam_results
24. expenses
25. expenses_vehicle
26. expenses_instructor
27. expenses_employee
28. expenses_general
29. instructor_price
30. notifications
31. instructor_schedule_publications
32. settings
33. transport_trips
34. transport_registrations
35. transport_attendance
36. activity_log

Transport can be delayed because it is prepared for future scope.

11. Key Decisions in V3

1. The system uses RBAC through roles, permissions, role_permissions, and user_roles.
2. User profile creation must be transactionally linked to role assignment.
3. users.password was renamed to users.password_hash.
4. training_type remains only MANUAL or AUTOMATIC.
5. lesson_price includes vehicle_source.
6. booking keeps PENDING_PAYMENT and EXPIRED to support simple hold logic and abuse prevention.
7. replaced_booking_id is unique.
8. booking_cancellations is 1:1 with booking.
9. booking_cancellations includes cancelled_by_user_id to know who performed the cancellation.
10. instructor_unavailable_periods uses start_at and end_at instead of day-only vacations.
11. vehicle_unavailable_periods is used instead of maintenance-only vehicle blocking.
12. shamcash_transactions stores transaction snapshot data.
13. student_charges includes certificate_exam_result_id.
14. student_payments allows multiple payments per charge.
15. expenses.created_by_employee_id identifies who recorded the expense.
16. certificates uses detailed workflow statuses.
17. certificates includes personal_photo_url.
18. certificate_training_sessions has UNIQUE(certificate_id, session_number).
19. certificate_exam_results keeps ABSENT.
20. notifications was added with a simple structure.
21. auth_otp_codes was added for OTP flows.
22. auth_sessions was added for refresh-token sessions.
23. instructor_schedule_publications was added for published instructor schedules.
24. settings remains the central table for configurable values.
25. Transport tables remain prepared for future scope.